Learning to Recover: A Controlled Study of Execution-Feedback Training for Small Coding Models

A matched-budget QLoRA comparison on function-level Python repair

[Author Name]
[Institution] `[contact or GitHub URL]`

November 2026 | Final frozen-holdout results included

Abstract

We ask whether a small coding language model learns to *recover* from failed code more effectively when fine-tuned on failure–execution-feedback–correction examples than when trained on successful code alone. Using Gemma 4 E4B-it, we collect visible-test trajectories and independently train two 4-bit QLoRA adapters: success-only (C1) and recovery-aware (C2), each with 55 examples and exactly 5,963 supervised completion tokens. The unmodified base model (C0) provides an additional control. In a paired, fixed-starting-state development experiment, C2 repairs 5/29 officially incorrect states (17.2%), compared with 4/29 (13.8%) for C1. **This advantage does not replicate on a frozen 70-task MBPP+ holdout:** among 24 officially incorrect, visible-failing initial states, C1 and C2 each repair 2 (8.3%), while C0 repairs 3 (12.5%). On a 30-task development study allowing up to three attempts, all conditions return officially correct final code on 16/30 tasks. A small feedback ablation favors detailed errors over a generic retry instruction (5/24 versus 3/24 for C2), but is inconclusive. We identify disagreement between original visible tests and the official extended evaluator, including false failure signals capable of inducing harmful edits. Our results do not establish a recovery advantage from tiny offline supervised datasets; they motivate broader recovery supervision and explicit auditing of feedback quality and final returned code.

1 Introduction

Agentic coding systems need to do more than generate a first answer: they must interpret failed tests, correct their code, and stop before further revisions cause regressions. Tool-augmented recovery is especially consequential for small models with limited compute budgets. Offline supervised fine-tuning on successful solutions is straightforward, but its targets contain no explicit failed attempt or execution feedback. Recovery-oriented training instead conditions a corrected solution on the model's preceding error and the observable test result.

We study whether that additional supervision produces a measurable benefit *under a small, matched training budget.* This is motivated by prior work finding that offline correction traces need not translate into self-correction behavior [3], while execution-feedback RL can improve iterative synthesis [4]. We restrict scope to standalone Python function synthesis and revision, rather than repository-level bug localization, file editing, or SWE-bench. The contribution is a transparent, reproducible controlled pilot, not a new optimization algorithm.

Our analysis distinguishes (a) visible-test failure, which triggers the agent's retry; (b) official MBPP+ correctness, used only for offline evaluation; and (c) final returned-code correctness, which can be lower than the oracle score of any attempt encountered. We report both development and independently frozen holdout results, including null findings.

2 Research Questions

RQ1: Does recovery-aware QLoRA outperform success-only QLoRA on identical failed starting states? **RQ2:** Are benefits concentrated in wrong-output, syntax, or runtime failures? **RQ3:** Does the recovery condition avoid repeated failed attempts or reduce tokens and test executions? **RQ4:** Does a benefit persist under a small retry budget and on held-out tasks or a separate programming benchmark?

3 Methods

3.1 Problem pool and trajectory collection

MBPP provides natural-language function specifications, reference implementations, and original visible tests [1]; the EvalPlus framework supplies more extensive scoring for MBPP+ and HumanEval+ [2]. We exclude 108 overlapping MBPP+ IDs from an initial 374-task MBPP training partition, retaining 266 eligible training problems. We sample 40 problems, generate two independent C0 trajectories each, and permit at most three attempts

using the original public tests. Across 80 trajectories, 53 first attempts pass visible tests; nine of 27 initially unsuccessful trajectories subsequently pass. The initially failing attempts comprise 22 wrong-output, four syntax, and one runtime failure. The dataset also includes 17 verified-reference corrections. All such verification is based on original visible tests and is not proof of extended-test correctness.

3.2 Training conditions

C0: The original instruction-tuned Gemma 4 E4B-it checkpoint [9]. **C1:** QLoRA trained on 50 successful model solutions and five reference solutions. **C2:** QLoRA trained on 27 successful model solutions, four standalone references, seven actual model self-recoveries, and 17 reference-based corrections. In a recovery item, the prompt includes the task, earlier unsuccessful code, and observable feedback; the target is corrected Python code. The prompt is masked from the loss. C1/C2 independently start from the same frozen C0; C2 is not further tuned from C1.

We use NF4 quantization, FP16, LoRA rank 8, scaling factor 16, `q_proj/v_proj` language-model targets, learning rate 10^{-4}, 1,024-token maximum sequence length, batch size one, four gradient-accumulation steps, and one epoch (14 optimizer updates per condition). The training seed is 42. Each condition contains 55 items and exactly 5,963 supervised target tokens, but input prompt length and source mixture differ. This controls target-token supervision, *not* total FLOPs or informational content. Training took 54.4 s (C1) and 73.9 s (C2) on one NVIDIA T4, excluding model setup and evaluation [7, 8].

3.3 Evaluation and metrics

We partition 180 distinct MBPP+ IDs into 110 development tasks (30 pilot plus 80 additional) and 70 final tasks. The final 70 IDs are recorded in the Day 5 manifest before their outcomes are inspected. The base model generates the *initial* code for each problem. States failing local visible tests form a paired *fixed-state* cohort: C0, C1, and C2 independently receive exactly the same starting code and feedback, then produce one correction each. The original visible tests are the only execution feedback used for generation. Official EvalPlus tests are evaluated in a separate stage, *after all completions are frozen*, and never provided to the generator.

The operational correction cohort contains all *visible-failing* starting states. For our audited metric we additionally restrict *post hoc* to the subset that fails official MBPP+ scoring initially, avoiding counting an already correct program as a repaired one. This restriction changes only the analysis denominator, not the model's input, action, or evaluated task IDs. We define repair success as the resulting code passing official MBPP+ tests. Differences between C2 and C1 are measured per paired problem; confidence intervals use percentile bootstrap resampling of tasks (20,000 draws). A second, exploratory 30-task development experiment measures up to three sequential attempts and counts both *ever-correct* and *final returned-code* outcomes. All models use a 384-token generation cap per attempt.

Table 1: Official MBPP+ one-step repair rates on the same C0 failed states. Officially correct initial code is excluded only in the audited analysis.

Split	C0	C1	C2
Development (29)	4/29	4/29	5/29
Frozen holdout (24)	3/24	2/24	2/24

4 Results

4.1 RQ1: Paired recovery and independent holdout

On development, 31 of 110 initial solutions fail visible tests. Two (IDs 139 and 276) nevertheless pass the official evaluator, leaving 29 audited incorrect states. C2 repairs five of these, while C1 and C0 each repair four. The C2–C1 difference is +3.45 percentage points (95% task-paired bootstrap interval $[0, 10.34]$ points); one task accounts for the gain.

On the frozen 70-task holdout, 35 C0 initial answers (50.0%) pass official MBPP+. Twenty-seven first attempts fail the local visible checks, but three (IDs 87, 233, 312) already pass official tests, leaving 24 genuinely incorrect fixed starting states. The **primary holdout comparison is a tie**: C1 and C2 each repair 2/24 (8.3%), while C0 repairs 3/24 (12.5%). C1 and C2 succeed on exactly the same two problems; their empirical paired bootstrap difference and interval are both zero. This degenerate interval describes this sample, *not proof of population equivalence*. The final notebook scored 151 generated solutions (70 initials and 27×3 corrections). No model parameters or prompt choices were updated from the frozen holdout results.

4.2 RQ2: Failure categories

Development failed states are concentrated in wrong-output cases: 29/31 visible failures are classified as wrong output and only two as syntax errors; none are runtime failures in this subset. C2 repairs 7/29 wrong-output cases when counting all visible-failing states, compared with 6/29 for C0 and C1. No condition repairs either of the two syntax cases. These descriptive counts are too small to attribute effects reliably to failure categories. The

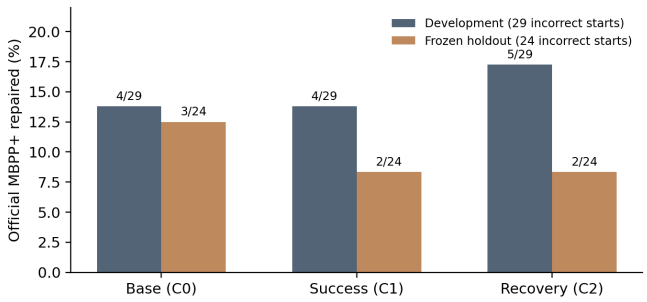


Figure 1: Paired fixed-state repair, audited against official MBPP+ scoring. The apparent one-problem development advantage of C2 does not replicate on the frozen holdout.

classification is derived from the local runner's observed feedback, not a hand-labeled semantic bug taxonomy.

4.3 RQ3–RQ4: Efficiency, retries and transfer

In the 30-task development end-to-end study, C0, C1, and C2 each return officially correct *final* code on 16/30 tasks. Counting any officially passing attempt instead yields 16/30, 17/30, and 17/30: both fine-tuned conditions sometimes revise a correct intermediate answer into an incorrect final one. Total output tokens are 6,086 (C0), 5,280 (C1), and 6,147 (C2). Normalized by *final* solved count, this is 380.4, 330.0, and 384.2 generated tokens per solved problem. Mean visible test executions per task are 1.50, 1.47, and 1.53. Hence the data do not show a C2 efficiency advantage. Exact repeated unsuccessful corrections in a separate fixed-state development comparison number 4, 3, and 3 for C0/C1/C2.

For exploratory transfer, first-attempt HumanEval+ correctness on 20 tasks is 11/20 (C0), 11/20 (C1), and 12/20 (C2). This is *not* a transfer-of-recovery benchmark, because no correction step is evaluated, and the sample is too small for a robust generalization claim. The frozen final experiment tests one fixed-state correction, not multi-attempt sequential recovery or cross-benchmark transfer.

4.4 Feedback specificity and reliability

On 24 newly added development failure states, C2 repairs 5 with detailed local test feedback versus 3 with only a generic retry instruction, a paired +8.3 percentage-point difference (95% bootstrap interval $[0, 20.8]$). C1 was not evaluated under this same ablation, so this does not isolate an effect of recovery training.

Local and official tests disagree in both splits (Table 2). On development, 17 initially visible-passing solutions fail the official tests and two initially visible-failing solutions pass them; on the frozen holdout, the corresponding counts are 11 and three. A separate reference-compatibility probe flags one reference implementation (holdout ID 123) with a local runtime/resource-limit error; we keep all 70 IDs rather than silently excluding it. These cases suggest that test coverage, equality semantics, and runner constraints deserve explicit checking. In particular, a false local failure signal can trigger a harmful edit, as seen in the end-to-end regression on development task 436.

Table 2: C0 first-attempt visible-test agreement with official MBPP+ scoring. Rows are the local test outcome.

Split	Local result	Official pass	Official fail
Development	Pass	62	17
Development	Fail	2	29
Final holdout	Pass	32	11
Final holdout	Fail	3	24

5 Discussion and Limitations

The simplest interpretation is *no detected benefit* from recovery-aware SFT with this small dataset, rather than a claim that learning recovery is impossible. Only seven C2 items are genuine model self-recoveries; the remaining recovery targets come from reference solutions. C2 also replaces some plain successes with correction-conditioned cases relative to C1. Such mixture changes, combined with 14 optimizer updates, a single seed, and prompt-length differences, limit causal attribution. The repaired-state cohorts (29 development and 24 final) are small; a one-case difference shifts the observed rate by 3.4–4.2 percentage points. Because starting states are sampled from the same base model, this experiment does not measure recovery from each fine-tuned model's *own* error distribution, an important distribution-shift caveat discussed by SCoRe [3].

The paper does not claim repository-scale software engineering capability. Original MBPP tests are imperfect proxies for correctness, and the subprocess runner has CPU/address-space limits but is *not* a security sandbox. We cannot exclude pretraining contamination by task-ID split alone. Moreover, the degenerate holdout paired CI arises from identical observed successes and offers limited uncertainty characterization outside this sample. We did not try extra epochs, change the dataset after holdout inspection, or train on holdout outcomes.

Promising future work includes collecting hundreds of independently verified *self*-recoveries, checking candidate corrections against stronger tests before supervised training, balancing correction and success mixtures, explicitly supervising diagnosis or root-cause localization, and comparing additional training epochs with multiple seeds. Such experiments should be tuned on new development data and evaluated on a *new* untouched holdout, not

reused to claim fresh independent confirmation from these 70 tasks. Recent negative-trajectory training research motivates better error attribution rather than treating all failed examples as equally informative [6].

6 Conclusion

A matched-budget comparison of success-only and recovery-aware QLoRA for Gemma 4 E4B-it found a one-case development improvement but **no improvement in the independently frozen holdout**. Both fine-tuned conditions repaired 2/24 officially incorrect initial states, and all three conditions returned the same number of correct final programs in the exploratory multi-turn development study. The most actionable lessons are that offline recovery data must be diverse and validated, paired evaluations must control the failed starting state, and a coding agent should be evaluated on its *final* code under reliable feedback. The accompanying notebooks and scripts reproduce the low-resource study and explicitly document these negative results.

Artifact availability

Source, notebooks, manifests, audited summaries, and figure scripts are supplied in the accompanying project repository; the public GitHub URL and Kaggle versioned artifact links must be filled in by the author before submission. Raw model-generated code and adapters should be published separately, if permitted, to enable complete independent replay.

References

[1] J. Austin et al. *Program Synthesis with Large Language Models*. arXiv:2108.07732, 2021.

[2] J. Liu, C. S. Xia, Y. Wang, and L. Zhang. *Is Your Code Generated by ChatGPT Really Correct? Rigorous Evaluation of Large Language Models for Code Generation*. NeurIPS, 2023.

[3] A. Kumar et al. *Training Language Models to Self-Correct via Reinforcement Learning*. ICLR, 2025.

[4] J. Gehring et al. *RLEF: Grounding Code LLMs in Execution Feedback with Reinforcement Learning*. ICML, 2025.

[5] N. Shinn et al. *Reflexion: Language Agents with Verbal Reinforcement Learning*. NeurIPS, 2023.

[6] M. Kim. *How to Learn from Negative Trajectories in Coding Agents*. Microsoft Debug-gym Research Blog, August 2026. `https://microsoft.github.io/debug-gym/blog/2026/08/negative-pi/`.

[7] E. J. Hu et al. *LoRA: Low-Rank Adaptation of Large Language Models*. ICLR, 2022.

[8] T. Dettmers et al. *QLoRA: Efficient Finetuning of Quantized LLMs*. NeurIPS, 2023.

[9] Google DeepMind. *Gemma 4 Model Card*, 2026. `https://ai.google.dev/gemma/docs/core/model_card_4`.